

딥레이서 레인 플레이북

팀원 4명이 오늘 밤 바로 실행할 수 있는 레인별 설정과 보상 함수. 트랙 자료는 실제 존재를 확인한 것만 실었고, 확인 결과 **학습된 모델 가중치는 공개된 것이 없다** — Import 전략을 그에 맞게 수정했다.

01 Import 전략 정정 — 남의 모델이 아니라 우리 모델을 옮긴다

먼저 정정합니다. 공개 저장소를 조사한 결과, DeepRacer 커뮤니티가 공유하는 것은 **보상 함수·트랙 waypoint·레이싱 라인 계산 코드**이고 **학습된 신경망 가중치는 사실상 공개되지 않습니다.** 실제 우승 모델 저장소들도 열어보면 `reward_function.py` 만 있습니다. "검증된 외부 모델을 import해서 출발한다"는 계획은 성립하지 않습니다.

대신 Import의 진짜 가치가 여기 있습니다. 계정이 4개이므로 **한 사람이 만든 좋은 체크포인트를 나머지 3명이 import해서 각자의 출발점으로 쓸 수 있습니다.** 밤마다 4명이 각자 처음부터 완주를 가르치는 낭비를 없애고, Gate 3에서는 승자 모델을 4계정으로 복제해 변형 4개를 동시에 돌릴 수 있습니다. 이것이 다른 팀이 갖지 못한 우리만의 구조적 이점입니다.

오늘 밤 순서 — 베이스 하나를 만들어 공유

1. 가장 먼저 끝나는 사람의 **Phase 1 모델**을 베이스로 지정한다. 조건: 5회 평가 100% 완주
2. 그 모델을 export → 나머지 3명이 **Import model**로 각자 계정에 올린다
3. 각자 그 모델을 **Clone**해서 자기 레인 보상 함수로 갈아끼우고 Phase 2를 시작한다

Import 시 확인할 것. 모델을 import하면 콘솔이 트랙을 항상 "reInvent 2018"로 표시하는 알려진 표시 버그가 있습니다 — 실제 학습 트랙과 무관하며 성능에 영향은 없습니다. 반면 **액션 스페이스와 센서 구성(모노 카메라, LiDAR 없음)**은 반드시 일치해야 하므로, 팀 전체가 **같은 센서 설정**을 쓰고 액션 스페이스만 레인별로 바꾸세요.

02 확인된 공개 자료 — 이걸 실제로 있습니다

아래는 존재와 내용을 직접 확인한 것만 실었습니다. 핵심은 우리 트랙의 최적 레이싱 라인이 이미 계산되어 공개돼 있다는 사실입니다.

① re:Invent 2018 트랙 waypoint 확인 완료

raw.githubusercontent.com/aws-solutions-library-samples/guidance-for-training-an-aws-deepracer-model-using-amazon-sagemaker/master/log-analysis/tracks/reinvent_base.npy

`reinvent_base` 가 곧 re:Invent 2018 트랙의 내부 이름입니다. shape (119, 6) — 119개 waypoint × 6열(중앙선 x·y, 내측 경계 x·y, 외측 경계 x·y). 레이싱 라인을 직접 계산하거나 로그 분석 시 트랙을 그릴 때 쓰는 원본 데이터입니다.

② 미리 계산된 K1999 최적 레이싱 라인 확인 완료 · 최우선

github.com/cdthompson/deepracer-k1999-race-lines → [racelines/reinvent_base-400-4-2019-10-11-161903.py](#)

우리 트랙용 최적 궤적이 이미 계산돼 있습니다. `[x, y]` 좌표 73개로 된 닫힌 루프이며, 파일을 열어 배열을 그대로 복사해 레인 B 보상 함수에 붙이면 됩니다. 첫 점은 `[2.88738855, 0.72646774]` 이고 마지막 점이 첫 점으로 돌아옵니다. `.npy` 버전도 함께 있습니다.

③ 레이싱 라인 계산 노트북 (직접 다시 계산할 때) 참고

github.com/cdthompson/deepracer-k1999-race-lines → [Race-Line-Calculation.ipynb](#)

K1999 알고리즘으로 waypoint에서 최적 궤적을 산출합니다. ②가 이미 있으니 필수는 아니지만, 반복 횟수를 바꿔 더 공격적인 라인을 뽑아보고 싶을 때 씁니다.

④ 구간별 최적 속도 & 액션 스페이스 계산 레인 C에 유용

github.com/dgnzlj/Capstone_AWS_DeepRacer → [Compute_Speed_And_Actions/RaceLine_Speed_ActionSpace.ipynb](#)

레이싱 라인의 각 점에서 곡률 기반 최대 속도를 계산해 `(x, y, speed, 예상시간)` 을 만들고, 거기서 액션 스페이스를 역산합니다. 레인 C의 목표 속도 테이블을 감이 아니라 계산으로 정하고 싶을 때 쓰세요.

⑤ 실차 우승 보상 함수 레퍼런스 (re:Invent 2018) 코드만

github.com/poPONUTS/aws-deepracer-model → [reward_function.py](#)

re:Invent 2018 트랙에서 시뮬 3위·실차 1위를 기록한 보상 함수입니다. 액션 스페이스는 속도 **1.1–2.0 m/s**, 조향 **±30°**, 랩타임 11초대. 모델 가중치는 없고 코드만 있습니다. 우리 설정을 검토할 때 비교 기준으로 유용하며, 실차 우승 모델의 최고 속도가 **2.0 m/s**였다는 점은 특히 참고할 만합니다.

⑤가 주는 **힌트**. 실차에서 이긴 설정의 상한이 2.0 m/s였다는 건, 우리가 계획한 2.4~2.6 m/s가 실차에서 버티지 못할 가능성을 시사합니다. 그래서 Day 2 오후의 속도 천장 실측이 결정적입니다 — 시뮬 숫자를 믿지 말고 재세요.

03 콘솔 설정 전체 가이드 — 5단계 워저드 따라가기

처음 하는 사람도 그대로 따라 할 수 있게 모든 입력 칸을 순서대로 설명합니다. 각 항목은 *이게 무엇인가 → 우리는 뭘 넣는가 → 왜 순으로 읽으세요*. 레인별로 다른 것은 액션 스페이스(Step 3)와 보상 함수(Step 5)뿐이고, 나머지는 4명 전원 동일하게 맞춰야 Import로 모델을 주고받을 수 있습니다.

Step 1 · Specify model name and environment

입력 칸	우리 값	설명 & 이유
Model name	A-p1-v1 형식	모델 이름. {레인} - {페이지} - v{번호} 규칙을 반드시 지킵니다. 공백·밑줄은 못 쓰고 하이픈만 됩니다. 나중에 40개 모델 중에서 뭐가 뭔지 찾으려면 이름이 유일한 단서입니다.
Race type	Time trial	경기 방식. 본선이 타임 트라이얼이므로 그대로 둡니다. Object avoidance는 장애물 회피용이라 우리와 무관합니다.
Environment simulation	re:Invent 2018 (A to Z 확정 시 변경)	학습할 트랙. 62개 목록에서 검색창에 reInvent 또는 A to Z 를 입력해 찾으세요. 두 트랙은 모양이 같고 폭만 다릅니다 — 아래 「먼저 확인할 것」을 읽고 실제 트랙 폭을 확인하세요. 확인 전에는 좁은 쪽(re:Invent 2018, 17.6m·76cm)으로 학습합니다. (레인 D만 Phase 1을 Oval Track에서 시작)
Track direction	팀 전체 통일	주행 방향(시계/반시계). 본선 방향을 확인해 그것과 같게 맞추세요. 방향이 다르면 코너 순서가 반대라 학습이 무의미해 집니다. 확인 전이라면 일단 팀 전체가 같은 방향으로만 통일하세요.
Minimum evaluation trials	5	학습 중 자동으로 돌리는 평가 주행 횟수. 우리 Gate 판정 기준이 "5회 연속 100% 완주"이므로 5로 둡니다. 낮추면 학습은 빨라지지만 모델이 진짜 안정적인지 알 수 없습니다.

Step 2 · Choose vehicle sensors and hyperparameters

센서는 4명 전원 똑같이. Camera (모노) 선택, Stereo camera 선택 안 함, LIDAR sensor 체크박스 해제. 실차가 카메라 1대·LiDAR 없음이므로 시물도 같아야 하고, 이게 다르면 팀 간 Import가 실패합니다. 기본 값이 이미 이 상태이니 **건드리지 마세요**.

알고리즘은 PPO를 선택합니다(기본값). SAC는 탐색을 더 하는 대신 수렴이 느려서 2박 3일에는 맞지 않습니다.

하이퍼파라미터 — 7개 값의 의미

Hyperparameters 접힌 부분을 펼치면 나옵니다. 기본값 그대로 두면 안 되는 값이 두 개(Epochs, Discount factor) 있습니다.

항목	기본값	우리 값	이게 무엇이고 왜 그렇게 두는가
Gradient descent batch size	64	64 유지	정책을 한 번 수정할 때 한꺼번에 참고하는 경험 데이터의 묶음 크기입니다. 크면 판단이 안정적이지만 느리고, 작으면 빠르지만 들쭉날쭉합니다. 64가 검증된 표준값이라 바꿀 이유가 없습니다.
Number of epochs	10	3 반드시 변경	모아둔 같은 경험 데이터를 몇 번 반복해서 되새김질할지입니다. 10번이면 같은 주행 기억을 열 번 복습하는 셈이라, 그 상황에만 지나치게 맞춰진 정책(과적합)이 됩니다. 그러면 새로운 코너에서 튕겨나갑니다. 초보자가 가장 많이 놓치는 값이고, 3으로 낮추는 것만으로 결과가 달라집니다.
Learning rate	0.0003	0.0003 → 0.0001	한 번 배울 때 운전 습관을 얼마나 크게 바꿀지입니다. 크면 빨리 배우지만 이미 잘하던 것도 부숩니다. Phase 1(처음 학습) 은 0.0003으로 빨리 배우게 하고, Clone 이후(Phase 2·3) 는 0.0001로 낮춰 잘 하던 걸 망가뜨리지 않고 다듬습니다.
Entropy	0.01	0.01 → 0.005	새로운 행동을 무작위로 시도하는 정도입니다. 높으면 이것저것 실험하고(초반에 필요), 낮으면 아는 방법을 반복합니다(후반 정밀 제어에 필요). Phase 1 은 0.01, Clone 이후 는 0.005로 낮춥니다.
Discount factor	0.99	0.995 변경	미래의 보상을 얼마나 중요하게 볼지입니다. 대략 $1/(1-\text{값})$ 스텝만큼 앞을 봅니다 — 0.99는 약 100스텝, 0.995는 약 200스텝 . DeepRacer가 초당 약 15스텝이니 0.995면 13초 앞, 거의 한 랩 전체 를 보는 셈입니다. "지금 감속해서 손해를 보지만 코너를 통과해 이득을 본다" 같은 판단을 배우려면 이 정도가 필요합니다. 0.999까지 올리면 시야는 넓어지지만 학습이 너무 느려집니다.
Loss type	Huber	Huber 유지	예측이 크게 틀렸을 때 얼마나 격하게 반응할지입니다. Mean squared error는 큰 실수 하나에 과민하게 반응해 학습이 흔들리고, Huber는 완만하게 받아들이어 안정적입니다. 이 콘솔은 이미 Huber가 기본값입니다.
Number of experience episodes between each policy-updating iteration	20	20 유지	정책을 한 번 수정하기 전에 주행 경험을 몇 판 모을지입니다. 적으면 자주 고치니 빠르지만 불안정하고, 많으면 신중하지만 느립니다. 20이 표준입니다.

Step 3 · Define action space **레이벌로 다룰**

차가 고를 수 있는 **(조향각, 속도) 조합의 목록**입니다. 랩타임을 좌우하는 가장 중요한 설정입니다.

콘솔은 두 가지를 제공합니다. **Continuous**는 좌·우 조향 한계와 최소·최대 속도, 이 네 숫자만 주고 차가 그 사이 아무 값이나 고르게 하는 방식입니다. 간단하지만 "큰 조향각에는 낮은 속도"라는 규칙을 강제할 수 없어서 차가 30° 조향에 최고 속도를 붙이는 조합도 자유롭게 고를 수 있고, 배울 것이 많아 수렴이 느립니다. **Discrete**는 우리가 정한 조합만 고르게 합니다. **2박 3일에는 Discrete를 선택하세요** — 기본값이 Continuous이므로 반드시 바꿔야 합니다.

Discrete 입력 절차 — 실제 화면 순서 **화면 확인 완료**

Discrete를 고르면 표를 직접 채우는 화면이 아니라, **격자를 자동 생성하는 화면**이 먼저 나옵니다. 여기서 **Advanced configuration**을 켜야 우리 표대로 값을 하나씩 넣을 수 있습니다. 순서는 이렇습니다.

1. **Discrete action space** **라디오 버튼 클릭** — 오른쪽 카드입니다(왼쪽 Continuous가 기본 선택돼 있음).
2. **Steering angle granularity = 5, Maximum steering angle = 30**으로 둡니다.
3. **Speed granularity = 2**로 둡니다. → 아래 **Action list** 에 **행 10개(0~9번)**가 자동 생성됩니다. 우리가 원하는 개수와 정확히 같습니다.
4. **Advanced configuration** **토글을 켭니다**(Action list 오른쪽 위). 그러면 각 행의 **조향각(-30~30)**과 **속도(0.1~4)**를 직접 입력할 수 있게 바뀝니다.
5. **10개 행의 값을 자기 레인 표대로 하나씩 고쳐 입력합니다**. 자동 생성된 기본값(-30°/0.5, -30°/1.0, ...)은 전부 덮어씁니다.
6. 행 개수를 조절할 때: **+ Add an action** 으로 추가(최대 20개까지 더), 행 오른쪽 **x** 로 삭제합니다. → **레인 C(12개)**는 2번 추가, **레인 D(8개)**는 2개 삭제하세요.
7. 오른쪽 **Radial polar graph** 로 확인합니다. **바깥쪽(큰 조향각) 화살표가 짧고, 위쪽(직진) 화살표가 가장 길면** 제대로 입력된 것입니다.

기본값 두 개가 함정입니다. 첫째, **Maximum speed 기본값이 1 m/s**입니다 — 이대로 두면 자동 생성 속도가 0.5·1.0뿐이라 랩타임이 20초를 넘습니다. 둘째, 자동 생성 격자는 **-30°에 최고 속도를 그대로 붙입니다**(예: 30°/1.0 m/s). 이 조합은 실차에서 스피드의 원인이므로, Advanced를 켜고 **큰 조향각 행의 속도를 반드시 낮춰야** 합니다. 격자를 그대로 쓰면 안 되는 이유가 바로 이것입니다.

Continuous를 쓸 수밖에 없다면(Advanced 입력이 너무 번거로운 경우) 차선택은 이렇습니다 — Left steering 22, Right steering -22, Minimum speed 1.4, Maximum speed 2.4. 조향 한계를 30°에서 22°로 좁혀 워블을 줄이고, 최소 속도를 1.4로 올려 차가 기어가지 못하게 막는 것이 요점입니다. 다만 각도·속도 결합을 강제할 수 없으니 보상 함수의 고속 과조향 페널티가 훨씬 중요해집니다.

액션 스페이스 설계 원칙 두 개. 첫째, 큰 조향각에는 반드시 낮은 속도를 매핑합니다(25° 조향에 2.4 m/s를 주면 실차는 반드시 스피ن합니다). 둘째, **최고 속도는 조향 0~5°에만 허용합니다**. 이 두 규칙을 어긴 액션 스페이스는 보상 함수를 아무리 잘 써도 살릴 수 없습니다. 개수는 8~12개가 적정이고, 20개를 넘기면 탐색 공간이 커져 2박 3일 안에 수렴하지 못합니다.

목표 랩타임에서 최고 속도를 역산하기

운영진에 따르면 **10초 이내 기록도 나옵니다**. 이 한 숫자로 우리가 넣어야 할 속도가 정해집니다. 코너에서 감속하기 때문에 평균 속도는 최고 속도의 **80~85%**가 됩니다 — 실차 우승 레퍼런스(자료 ⑤)가 최고 2.0 m/s에서 11초(평균 1.6 m/s)였으니 좁은 트랙에서 비율 0.8, 넓은 트랙은 코너를 더 빠르게 통과할 수 있어 0.85 정도로 봅니다.

목표 랩타임	re:Invent 2018 17.6m · 76cm	A to Z Speedway 16.64m · 107cm	판정
11초	최고 2.0	최고 1.8	보험용 (레인 A)
10초	최고 2.2	최고 2.0	최소 경쟁선
9초	최고 2.45	최고 2.2	입상권
8초	최고 2.75	최고 2.45	우승권

결론 — 최고 속도 2.0 m/s로는 10초 벽을 넘기 어렵습니다. 그래서 레인 B의 스위치를 **2.2 / 2.4 / 2.6**, 레인 C를 **2.4 / 2.6 / 2.8**로 올렸습니다. 레인 A만 2.0을 유지해 보험 역할을 맡습니다. 위 비율은 레퍼런스 한 건에서 나온 대략치이므로, **Day 2 오후에 실차 랩타임을 재서 우리 비율로 교체하세요.**

먼저 확인할 것 — 우리 트랙은 두 후보 중 어느 쪽인가

re:Invent 2018과 A to Z Speedway는 모양이 똑같습니다. A to Z Speedway는 콘솔 설명 그대로 "re:Invent 2018의 폭이 넓은 버전"이라서, **트랙 사진만으로는 절대 구분할 수 없습니다.** 다른 것은 폭과 길이 뿐입니다.

항목	re:Invent 2018	A to Z Speedway
길이	17.6 m	16.64 m
폭	76 cm	107 cm
내부 이름	reinvent_base	reInvent2019_wide
K1999 라인 공개 여부	있음 (73점, 자료 ②)	없음 — 직접 계산 필요
난이도	좁아서 탈선 쉬움	31cm 더 넓어 훨씬 관대

구분법은 간단합니다 — 줄자로 트랙 폭을 재세요. 76cm와 107cm는 눈으로도 확실히 다릅니다. 차 폭이 약 20cm이니, 차 3.8대가 들어가면 좁은 쪽(2018), 5.3대가 들어가면 넓은 쪽(A to Z)입니다.

확인이 안 될 때의 안전한 선택: 좁은 쪽(re:Invent 2018)으로 학습하세요. 76cm 안에서 달릴 수 있는 모델은 107cm에서도 문제없지만, 반대는 성립하지 않습니다. 게다가 좁은 쪽은 최적 레이싱 라인이 이미 공개돼 있습니다(자료 ②). 단 **A to Z로 확정되면 즉시 갈아타고 속도를 한 단계 올리세요** — 폭이 31cm 넓으면 훨씬 공격적으로 갈 수 있고, 운영진이 말한 10초가 더 쉽게 나옵니다.

레인별 구체적인 표는 아래 **04~07절**에 있습니다. 자기 레인 표를 그대로 입력하세요.

Step 4 · Choose a shell

차량 외형과 색상으로, 성능에는 전혀 영향이 없습니다(콘솔에도 "cosmetic only"라고 적혀 있습니다). 다만 **4명이 서로 다른 Trim color를 고르면** 학습 영상을 볼 때 누구 모델인지 바로 구분되니 그렇게 하세요.

Step 5 · Customize reward function

코드 편집기에 자기 레인의 보상 함수를 붙여넣습니다. 기본으로 들어있는 중앙선 예제 코드는 **전체 선택 후 지우고** 교체하세요.

- 붙여넣은 뒤 반드시 **Validate** 버튼을 누르세요. 오타나 문법 오류를 여기서 잡습니다. 이걸 건너뛰고 학습을 시작하면 45분을 날립니다.
- 편집기 하단에 **Errors: 0**, **Warnings: 0** 인지 확인합니다.
- **Stop condition · Maximum time: 45~60분**으로 설정합니다(기본값 10분은 너무 짧습니다). 최대 1440분까지 넣을 수 있지만 **절대 그러지 마세요** — 3시간을 넘기면 시뮬레이터의 배경 픽셀까지 암기해 실차에서 무너집니다.
- 마지막으로 **Train your model** 을 누릅니다.

보상 함수를 쓸 때 지켜야 할 규칙

규칙	이유
반환값은 항상 양수 — 최소 <code>1e-3</code>	0이나 음수를 반환하면 학습이 깨집니다. 탈선처럼 최악인 경우에도 <code>return 1e-3</code> 으로 아주 작은 양수를 줍니다.
반환값은 <code>float</code> — <code>return float(reward)</code>	정수나 다른 타입이 섞이면 오류가 납니다.
함수 이름은 정확히 <code>reward_function(params)</code>	이름이 다르면 인식되지 않습니다.
<code>params</code> 키 이름 오타 주의	<code>distance_from_center</code> , <code>all_wheels_on_track</code> , <code>closest_waypoints</code> , <code>steering_angle</code> , <code>track_width</code> , <code>is_offtrack</code> 등. 한 글자만 틀려도 학습이 통째로 무의미해집니다 — Validate 로 확인하세요.
덧셈 항과 곱셈 항을 섞지 않기	곱셈 기반 보상(0.3~2.0 범위)에 <code>reward += ... * 10</code> 처럼 큰 덧셈을 얹으면 덧셈 항이 전체를 지배해 조향·속도 페널티가 무력화됩니다.
한 번에 한 가지만 바꾸기	보상 함수와 액션 스페이스를 동시에 바꾸면 무엇이 효과였는지 알 수 없습니다.

초보자가 자주 하는 실수 7가지

1. **Epochs**를 기본값 **10**으로 두기 — 과적합의 1번 원인. 3으로 낮추세요.
2. 한 모델을 몇 시간씩 돌리기 — 오래 = 좋음 이 아닙니다. 45~60분씩 끊어 Clone하세요.
3. 액션 스페이스를 **Continuous**로 두기 — 기본값이므로 **Discrete**로 바꿔야 합니다.
4. **Discrete**만 고르고 **Advanced configuration**을 안 켜기 — 그러면 자동 생성 격자가 큰 조향각에 최고 속도를 붙인 채로 학습됩니다. 실차 스프인의 직행 코스입니다.
5. **Maximum speed**를 기본값 **1**로 두기 — 랩타임이 20초를 넘습니다. 레인별 값으로 올리세요.
6. 큰 조향각에 높은 속도 매핑 — 실차에서 100% 스프인합니다.

7. 보상 함수에 목표를 여러 개 넣기 — 중앙선 유지와 레이싱 라인 추종을 같이 넣으면 서로 싸워 수렴하지 않습니다.
8. **Validate** 안 누르고 학습 시작 — 오타 하나로 45분을 날립니다.
9. 시뮬 완주율 **100%**를 보고 **안심하기** — 실차에서 한 바퀴도 못 도는 일이 흔합니다. Day 2 오후 실차 테스트만이 진짜 검증입니다.

한 장 요약 — 이대로 입력하세요

Phase 1 (처음 학습)

트랙 re:Invent 2018 · Race type Time trial · Evaluation trials 5 · 센서 Camera만(LiDAR 해제) · 알고리즘 PPO

Batch 64 · **Epochs 3** · LR **0.0003** · Entropy **0.01** · **Discount 0.995** · Loss Huber · Episodes 20

액션 스페이스 **Discrete**(레인별 표) · Maximum time **45~60분**

Phase 2·3 (Clone 이후) — 위와 같고 두 개만 낮춥니다

LR **0.0003 → 0.0001** · Entropy **0.01 → 0.005**

A 중앙선 베이스라인 안전

검증할 가설: "확실히 완주하는 기록을 먼저 확보한다"

이 레인의 임무는 순위가 아니라 보험입니다. 본선 1차에 올릴 모델이며, 다른 셋이 모두 무너져도 유효 기록을 남깁니다. 절대 욕심내지 말고 완주 안정성만 봅니다. 팀에서 가장 먼저 5회 평가 100%를 달성하면 그 모델이 팀 공용 베이스가 되니, 이 레인을 최우선으로 끝내세요.

액션 스페이스 (이산 10개)

#	조향	속도	#	조향	속도
0	-25°	1.4	5	+5°	2.0
1	-15°	1.6	6	+10°	1.8
2	-10°	1.8	7	+15°	1.6
3	-5°	2.0	8	+25°	1.4
4	0°	2.0	9	0°	1.6

보상 함수

```
def reward_function(params):
    # [레인 A] 중앙선 유지 전용 - 속도 보상 없음(속도는 액션 스페이스로 통제)
    if params['is_offtrack'] or not params['all_wheels_on_track']:
        return 1e-3

    tw = params['track_width']
    d = params['distance_from_center']
    speed = params['speed']
    steer = abs(params['steering_angle'])

    reward = 1.0

    # 중앙선 근접도
    if d <= 0.10 * tw:
        reward *= 1.3
    elif d <= 0.25 * tw:
        reward *= 1.0
    elif d <= 0.40 * tw:
        reward *= 0.5
    else:
        reward *= 0.15

    # 고속 과조향 억제
    if speed > 1.6 and steer > 20:
        reward *= 0.7

    return float(reward)
```

진행 순서

1. 위 설정으로 45분 학습 → 5회 평가
2. 100% 완주면 **즉시 팀에 공유(export)**하고 알린다
3. 미달이면 액션 스페이스 속도를 전부 0.2 낮춰 재학습
4. 달성 후에는 이 모델을 클론해 속도만 조금씩 올려본다 (2.0 → 2.2)

B K1999 최적 궤적 추종 본진

검증할 가설: "최적 궤적이 랩타임을 만든다"

이 레인은 이미 계산된 답을 쓰기 때문에 가장 유리합니다. 자료 ②의 `reinvent_base-400-4-2019-10-11-161903.py`를 열어 73개 좌표 배열을 그대로 복사해 아래 `RACE_LINE`에 붙여넣으세요. 직접 계산할 필요가 없습니다.

액션 스페이스 (이산 10개 · 병렬 3개 스왑)

아래 표에서 `CAP` 한 칸만 **2.0 / 2.2 / 2.4**로 바꾼 세 개를 동시에 학습시킵니다. 한 칸만 다르므로 결과 차이가 곧 속도 효과입니다.

#	조향	속도	#	조향	속도
0	-24°	1.3	5	+5°	2.1
1	-16°	1.6	6	+10°	1.9
2	-10°	1.9	7	+16°	1.6
3	-5°	2.1	8	+24°	1.3
4	0°	CAP	9	0°	1.7

보상 함수

```

import math

# 자료 ②에서 복사 - cdthompson/deepracer-k1999-race-lines
# racelines/reinvent_base-400-4-2019-10-11-161903.py (73점, 달한 루프)
RACE_LINE = [
    [2.88738855, 0.72646774],
    [3.16759122, 0.70478649],
    [3.45517317, 0.69217863],
    # ... 파일의 나머지 좌표를 모두 붙여넣을 것 ...
    [2.61751276, 0.75992145],
]

def reward_function(params):
    # [레인 B] 최적 레이싱 라인 추종 + 진행 방향 정렬
    if params['is_offtrack'] or not params['all_wheels_on_track']:
        return 1e-3

    x, y = params['x'], params['y']
    tw = params['track_width']

    # 최근점 레이싱 라인 점까지의 거리
    dmin = min(math.hypot(x - px, y - py) for px, py in RACE_LINE)

    # 궤적 오차 보상 (트랙 폭으로 정규화 - 붙으면 1.0, 멀어지면 급감)
    reward = math.exp(-6.0 * (dmin / tw) ** 2)

    # 진행 방향 정렬
    wps = params['waypoints']
    n = len(wps)
    p0 = wps[params['closest_waypoints'][0]]
    p1 = wps[(params['closest_waypoints'][1] + 1) % n]
    track_dir = math.degrees(math.atan2(p1[1] - p0[1], p1[0] - p0[0]))
    diff = abs(track_dir - params['heading'])
    if diff > 180:
        diff = 360 - diff

    if diff <= 10:
        reward *= 1.2
    elif diff > 25:
        reward *= 0.6

    # 워블 억제
    if params['speed'] > 1.8 and abs(params['steering_angle']) > 15:
        reward *= 0.7

    return float(max(reward, 1e-3))

```

중요. 이 레인은 중앙선 보상을 일부러 넣지 않았습니다. 레이싱 라인은 코너에서 의도적으로 중앙선을 벗어나므로, 중앙선 보상을 함께 넣으면 두 목표가 싸워 수렴하지 않습니다.

진행 순서

1. 자료 ②에서 73개 좌표를 복사해 `RACE_LINE` 을 완성 (가장 먼저 할 일)
2. 팀 공용 베이스 모델을 import → Clone → 위 보상 함수로 교체
3. CAP 2.0 / 2.2 / 2.4 세 개를 동시에 60분 학습
4. 세 개 모두 5회 평가 → 완주율과 랩타임을 시트에 기록. **완주율이 무너지는 CAP이 곧 시물 상한**

C 곡률 lookahead 감가속 공격

검증할 가설: "궤적보다 감가속 타이밍이 랩타임을 만든다"

외부 데이터가 전혀 필요 없는 레인입니다. 앞쪽 waypoint의 방향 변화량으로 다가오는 코너의 급함을 실시간 계산하고, 그에 맞는 목표 속도를 추종하게 합니다. 직선에서 최대한 밀고 코너 진입 전에 미리 줄이는, 사람이 운전하는 방식이죠.

액션 스페이스 (이산 12개 · 병렬 3개 스왑)

최고 속도를 2.2 / 2.4 / 2.6으로 바꾼 세 개를 동시에 돌립니다. 보상 함수의 target 최댓값도 함께 맞춰야 합니다.

#	조향	속도	#	조향	속도
0	-28°	1.3	6	+4°	2.4
1	-20°	1.5	7	+8°	2.1
2	-14°	1.8	8	+14°	1.8
3	-8°	2.1	9	+20°	1.5
4	-4°	2.4	10	+28°	1.3
5	0°	CAP	11	0°	1.9

보상 함수

```

import math

def reward_function(params):
    # [레인 C] 앞 구간 곡률로 목표 속도를 정하고 그것을 추종
    if params['is_offtrack'] or not params['all_wheels_on_track']:
        return 1e-3

    wps = params['waypoints']
    n = len(wps)
    nx = params['closest_waypoints'][1]
    speed = params['speed']
    steer = abs(params['steering_angle'])
    tw = params['track_width']

    def ang(i, j):
        a, b = wps[i % n], wps[j % n]
        return math.atan2(b[1] - a[1], b[0] - a[0])

    # 앞쪽 두 구간의 방향 변화량 = 다가오는 코너의 급함
    K = 4
    delta = math.degrees(ang(nx + K, nx + 2 * K) - ang(nx, nx + K))
    while delta > 180:
        delta -= 360
    while delta < -180:
        delta += 360
    turn = abs(delta)

    # 곡률 → 목표 속도 (CAP을 바꾸면 아래 첫 값도 같이 바꿀 것)
    if turn < 5:
        target = 2.6
    elif turn < 12:
        target = 2.2
    elif turn < 25:
        target = 1.8
    else:
        target = 1.4

    # 목표 속도와 오차 (정확히 맞으면 1.0)
    reward = math.exp(-1.2 * abs(speed - target))

    # 트랙 유지는 약하게만 (궤적 자유도를 남긴다)
    if params['distance_from_center'] > 0.42 * tw:
        reward *= 0.4

    # 워블 억제
    if speed > 1.8 and steer > 15:
        reward *= 0.7

    return float(max(reward, 1e-3))

```

튜닝 포인트. K = 4 는 얼마나 앞을 보는지입니다. 코너 진입이 늦어 이탈하면 K 를 5~6으로 늘려 더 미리 감속하게 하고, 너무 일찍 느려지면 3으로 줄이세요. **한 번에 이 값 하나만** 바꿉니다.

진행 순서

1. 공용 베이스 import → Clone → 위 보상 함수로 교체
2. CAP 2.2 / 2.4 / 2.6 세 개 동시 학습 (60분)
3. 평가 후 이탈 지점을 로그로 확인 → K 만 조정해 재학습
4. 자료 ④의 노트북으로 목표 속도 테이블을 계산값으로 교체하면 한 단계 더 정밀해진다

D 워블 억제 + 일반화 해지

검증할 가설: "시물 1등이 실차 1등이 아니다"

Import가 외부 모델용으로는 쓸 수 없게 됐으므로, 이 레인은 원래 계획대로 **실차 생존에 특화**합니다. 조향 각도를 $\pm 20^\circ$ 로 좁히고 **조향 변화율**에 페널티를 걸어 실차에서 지그재그가 나지 않는 모델을 만듭니다. Day 2 오후에 B·C가 워블로 무너지면 **이 레인이 유일한 대안**입니다.

여력이 있으면 Phase 1을 **Oval Track**에서 돌린 뒤 대회 트랙으로 클론해, "다른 트랙 선행 학습이 일반화에 도움이 되는가"라는 질문의 대조군도 겸합니다.

액션 스페이스 (이산 8개 · 조향 $\pm 20^\circ$ 제한)

#	조향	속도	#	조향	속도
0	-20°	1.2	4	$+6^\circ$	1.8
1	-12°	1.5	5	$+12^\circ$	1.5
2	-6°	1.8	6	$+20^\circ$	1.2
3	0°	2.0	7	0°	1.4

보상 함수

```

import math

# 직전 스텝의 조향각을 기억 (조향 변화율 계산용)
_prev_steer = 0.0

def reward_function(params):
    # [레인 D] 실차 워블 억제 특화
    global _prev_steer

    steer = params['steering_angle']

    # 에피소드 시작 시 초기화
    if params['steps'] <= 1:
        _prev_steer = steer

    if params['is_offtrack'] or not params['all_wheels_on_track']:
        _prev_steer = steer
        return 1e-3

    tw = params['track_width']
    d = params['distance_from_center']
    speed = params['speed']

    # 중앙선 유지 (완만한 곡선형 보상)
    reward = math.exp(-4.0 * (d / tw) ** 2)

    # 조향 변화율 페널티 - 이 레인의 핵심
    rate = abs(steer - _prev_steer)
    if rate > 15:
        reward *= 0.5
    elif rate > 8:
        reward *= 0.8

    # 고속 과조향 억제
    if speed > 1.7 and abs(steer) > 15:
        reward *= 0.7

    _prev_steer = steer
    return float(max(reward, 1e-3))

```

주의 — 전역 변수 기법의 한계. 조향 변화율은 직전 스텝을 기억해야 하므로 모듈 전역 변수를 씁니다. 이는 커뮤니티에서 널리 쓰이는 방법이지만, 학습 워커가 여러 개일 때 값이 섞일 수 있습니다. 학습이 이상하게 수렴하면 **변화율 항을 빼고 절대 조향각 페널티만 남기세요** — 효과는 줄지만 안전합니다.

진행 순서

1. Phase 1을 **Oval Track**에서 45분 학습 (대조군 목적)
2. 대회 트랙으로 Clone → 위 보상 함수로 60분
3. 평가 후, 다른 레인과 **완주율 안정성**을 비교해 시트에 기록
4. Day 2 오후 실차 테스트에서 이 레인의 진짜 가치가 드러난다 — 랩타임이 느려도 흔들리지 않으면 승자가 될 수 있다

08 공용 기록 시트 — 학습 1회 = 1행

모델 이름은 {레인}-{페이지}-{v{번호}} 형식으로 통일합니다. 예: B-p2-v3 , C-p3-v1 .

모델명	레인	이전 버전에서 바꾼 것 <i>하나</i>	평가 완주율	평균 랩타임	실차 결과
A-p1-v1	A	(최초)	5/5	13.1s	—
B-p2-v1	B	CAP 2.0			
B-p2-v2	B	CAP 2.2			
B-p2-v3	B	CAP 2.4			

한 번에 하나만 바꿉니다. 보상 함수와 액션 스페이스를 동시에 바꾸면 무엇이 효과였는지 영원히 알 수 없습니다. 시트의 "바꾼 것 하나" 칸에 두 가지를 적고 있다면 그 실험은 이미 망친 것입니다.

09 오늘 밤 체크리스트

- ☐ 4명 전원 센서 설정 통일 (Camera 단독, LiDAR 해제) — Import 호환의 전제
- ☐ 액션 스페이스: **Discrete 선택** → **Advanced configuration 토글 ON** → 레인별 표대로 값 입력
- ☐ Epochs를 기본값 10 → **3**으로 변경 (전원)
- ☐ Discount factor 0.99 → **0.995** (전원)
- ☐ Maximum time 45~60분으로 설정, 절대 초과하지 않기
- ☐ 레인 B 담당: 자료 ②에서 73개 좌표 복사해 `RACE_LINE` 완성
- ☐ 레인 A 담당: 가장 먼저 5회 평가 100% 달성 후 export하여 팀 공유
- ☐ 공용 기록 시트 생성 및 링크 공유
- ☐ 취침 전 4계정 모두 다음 학습 예약 (병렬 슬롯 비우지 않기)

딥레이서 레인 플레이북 · 「딥레이서 우승 작전」 브리프의 실행 편 · 자료 ①~⑤는 2026-08-19 존재 확인